

최종 발표

PROJECT



논문 제목 생성기

Ab2Ti (Abstract to Title)

자연어 처리 | 미니프로젝트

2조

202204046 류동우

202204249 이민우

* Contents

1. 미니 프로젝트 주제
2. 사용한 데이터셋
3. 코드 구현
 - Google T5, 수정 코드
 - Dynamic Context Gates
4. 데모 시연

* 프로젝트 배경

논문을 보고, 연구하다 보면 드는 의문 → 어떻게 논문의 제목을 잘 지을까

1. [arXiv:2504.05226](#) [pdf, other] [cs.CL](#)

Proposing TAGbank as a Corpus of Tree-Adjoining Grammar Derivations

Authors: [Jungyeul Park](#)

Abstract: ...of lexicalized grammars, particularly Tree-Adjoining Grammar (TAG), has significantly advanced our understanding of syntax and semantics in natural language processing (NLP). While existing syntactic resources like the Penn Treebank and Universal Dependencies offer extensive annotations for phrase-structure and dependency parsing, there is a lack of large-sc... [▽ More](#)

Submitted 7 April, 2025; originally announced April 2025.

2. [arXiv:2504.04385](#) [pdf] [cs.CL](#)

Pre-trained Language Models and Few-shot Learning for Medical Entity Extraction

Authors: [Xiaokai Wang](#), [Guiran Liu](#), [Binrong Zhu](#), [Jacky He](#), [Hongye Zheng](#), [Hanlu Zhang](#)

Abstract: ...research can integrate knowledge graphs and active learning strategies to improve the model's generalization and stability, providing a more effective solution for medical NLP research. Keywords- Natural Language Processing, medical named entity recognition, pre-trained language model, Few-shot Learning, information extraction, deep learning [▽ More](#)

Submitted 6 April, 2025; originally announced April 2025.

3. [arXiv:2504.04275](#) [pdf, other] [cs.CL](#)

negativas: a prototype for searching and classifying sentential negation in speech data

Authors: [Túlio Sousa de Gois](#), [Paloma Batista Cardoso](#)

Abstract: ...four stages: i) analyzing a dataset of 22 interviews from the Falares Sergipanos database, annotated by three linguists, ii) creating a code using natural language processing (NLP) techniques, iii) running the tool, iv) evaluating accuracy. Inter-annotator consistency, measured using Fleiss' Kappa, was moderate (0.57). The tool identified 3,338 instances... [▽ More](#)

Submitted 5 April, 2025; originally announced April 2025.



* 우리가 집중했던 것은

Abstract

We introduce Clinical ModernBERT, a transformer-based encoder pretrained on large-scale biomedical literature, clinical notes, and medical ontologies, incorporating PubMed abstracts, MIMIC-IV clinical data, and medical codes with their textual descriptions. Building on ModernBERT [Warner et al., 2024]—the current state-of-the-art natural language text encoder featuring architectural upgrades such as rotary positional embeddings (RoPE), Flash Attention, and extended context length up to 8,192 tokens—our model adapts these innovations specifically for biomedical and clinical domains. Clinical ModernBERT excels at producing semantically rich representations tailored for long-context tasks. We validate this both by analyzing its pretrained weights and through empirical evaluation on a comprehensive suite of clinical NLP benchmarks.

Abstract



Clinical ModernBERT: An efficient and long context encoder for biomedical text

title

✱ 프로젝트 목표 설정

T5 모델을 기반으로 논문 초록으로부터
적절한 제목을 생성하는 파이프라인 구축.

"Dynamic Context Gate" 메커니즘을 도입하
여

T5 모델의 제목 생성 성능 개선 가능성 탐색.

* 데이터셋 : arXiv 데이터셋

arXiv Dataset

arXiv dataset and metadata of 1.7M+ scholarly papers across STEM



- `id` : ArXiv ID (can be used to access the paper, see below)
- `submitter` : Who submitted the paper
- `authors` : Authors of the paper
- `title` : Title of the paper
- `comments` : Additional info, such as number of pages and figures
- `journal-ref` : Information about the journal the paper was published in
- `doi` : https://www.doi.org (Digital Object Identifier)
- `abstract` : The abstract of the paper
- `categories` : Categories / tags in the ArXiv system
- `versions` : A version history

```
def preprocess_data(data, tokenizer):  
  
    inputs = []  
    targets = []  
  
    for item in data:  
  
        abstract = item["abstract"].strip()  
        title = item["title"].strip()  
  
        if len(abstract.split()) > 400:  
            abstract = ' '.join(abstract.split()[:400])  
  
        inputs.append(f"generate title: {abstract}")  
        targets.append(title)
```

arXiv는 다양한 연구자의 논문을 사전 공개해 빠르게 공유하고 피드백을 받는 플랫폼
170만 편 이상의 STEM(과학, 기술, 공학, 수학) 분야 논문의 메타데이터 / 1991년 ~ 현재

* 데이터셋 : arXiv 데이터셋

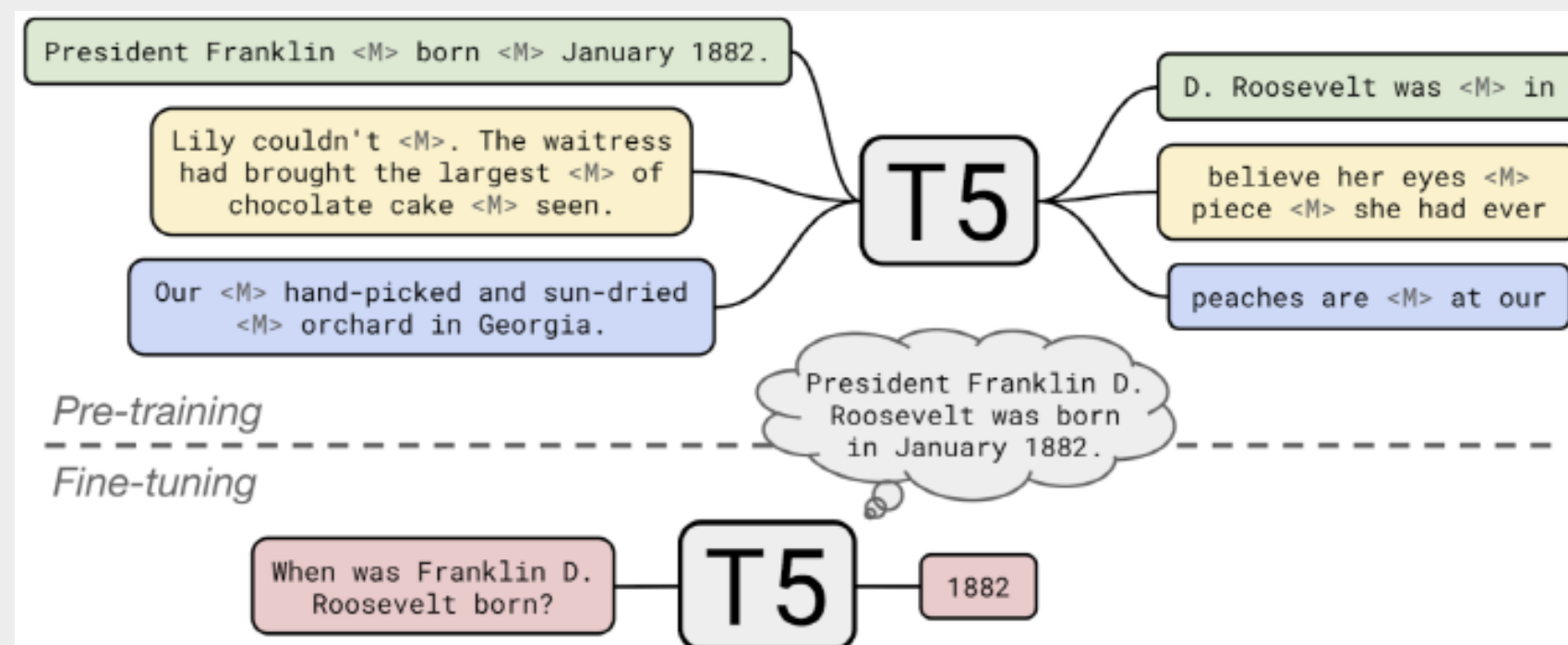
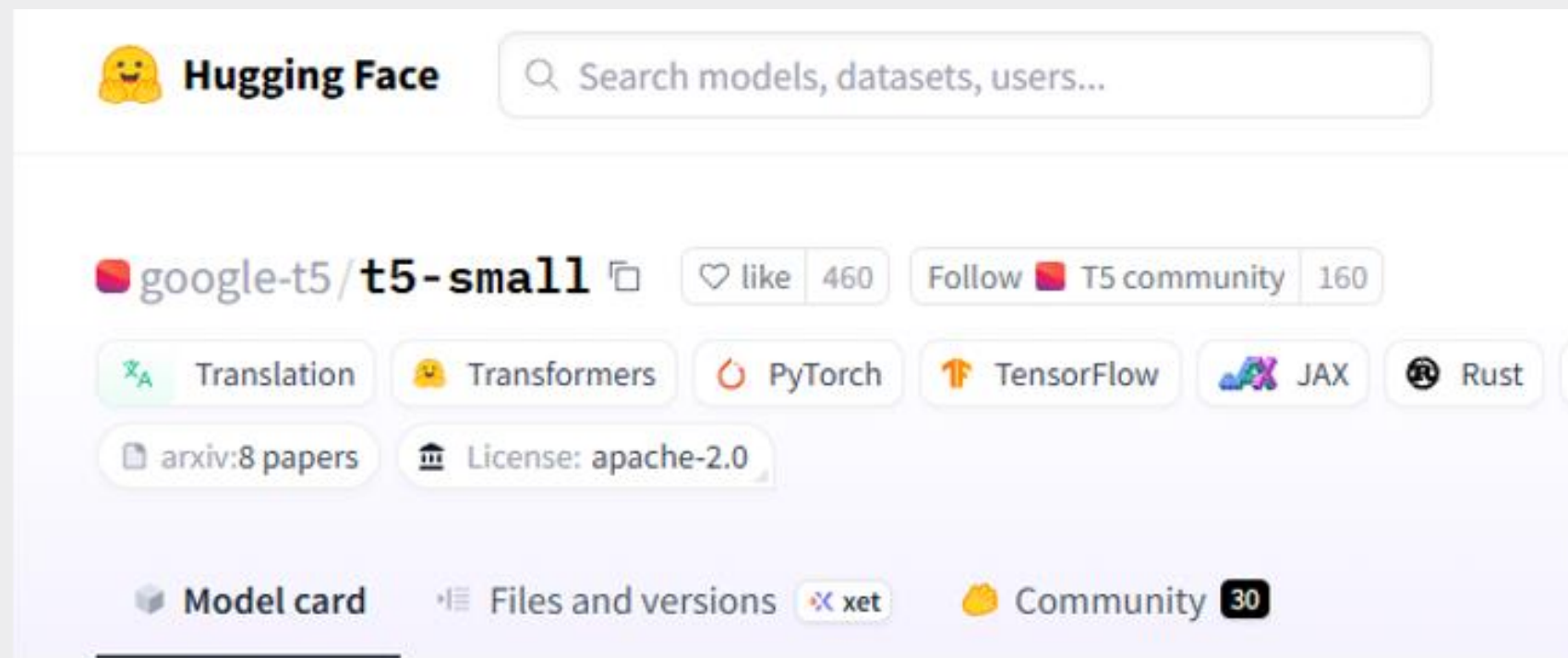
"title" :

string "Calculation of prompt diphoton production cross sections at Tevatron and LHC energies"

"abstract" :

string " A fully differential calculation in perturbative quantum chromodynamics is presented for the production of massive photon pairs at hadron colliders. All next-to-leading order perturbative contributions from quark-antiquark, gluon-(anti)quark, and gluon-gluon subprocesses are included, as well as all-orders resummation of initial-state gluon radiation valid at next-to-next-to-leading logarithmic accuracy. The region of phase space is specified in which the calculation is most reliable. Good agreement is demonstrated with data from the Fermilab Tevatron, and predictions are made for more detailed tests with CDF and D0 data. Predictions are shown for distributions of diphoton pairs produced at the energy of the Large Hadron Collider (LHC). Distributions of the diphoton pairs from the decay of a Higgs boson are contrasted with those produced from QCD processes at the LHC, showing that enhanced sensitivity to the signal can be obtained with judicious selection of events. "

* Google T5



1. 완전한 Text-to-Text Framework

T5 : 모든 NLP Task → text to text 포맷으로 통일
“초록 → 제목 생성”과 같은 파이프라인을
한 번에 다룰 수 있어 기본 구성요소 (임베딩→셀프어텐션
→피드포워드→디코더)를 모두 포함할 수 있다.

2. 사전학습(Transfer Learning)의 이점

논문 초록-제목 데이터는 보통 수십만 건 이상의 학습 데이터 필요,
T5-base는 이미 C4 코퍼스 같은 방대한 데이터로 학습되어 있어,
적은 데이터로도 빠르게 수렴하고 안정적인 결과를 얻을 수 있음.

* Config

```
CONFIG = {  
    "model_name": "t5-small",  
    "output_dir": "./model",  
    "data_dir": "./data",  
    "batch_size": 8,  
    "learning_rate": 3e-5, # DCG 모델과 동일하게 낮춤  
    "num_epochs": 5,  
    "max_input_length": 512,  
    "max_target_length": 128,  
    "train_size": 9000, # DCG 모델과 동일한 데이터  
    # 크기  
    "val_size": 1000,  
    "test_size": 1000,  
    "warmup_ratio": 0.1, # DCG 모델과 동일  
    "gradient_accumulation_steps": 2 # DCG 모델과  
    # 동일  
}
```

모든 모델은 다음 기본 설정으로 공정한 비교를 위해 실험

- 모델: t5-small
- 배치 크기: 8
- 학습률: $3e-5$
- 에폭: 5
- 입력 길이: 512 (초록)
- 출력 길이: 128 (제목)
- 훈련 데이터: 9000개 샘플
- 검증/테스트: 각 1000개 샘플

* Base T5

```
# base_train.py
def train_model():
    # 모델과 토크나이저 로드
    tokenizer = T5Tokenizer.from_pretrained(CONFIG["model_name"])
    model = T5ForConditionalGeneration.from_pretrained(CONFIG["model_name"])

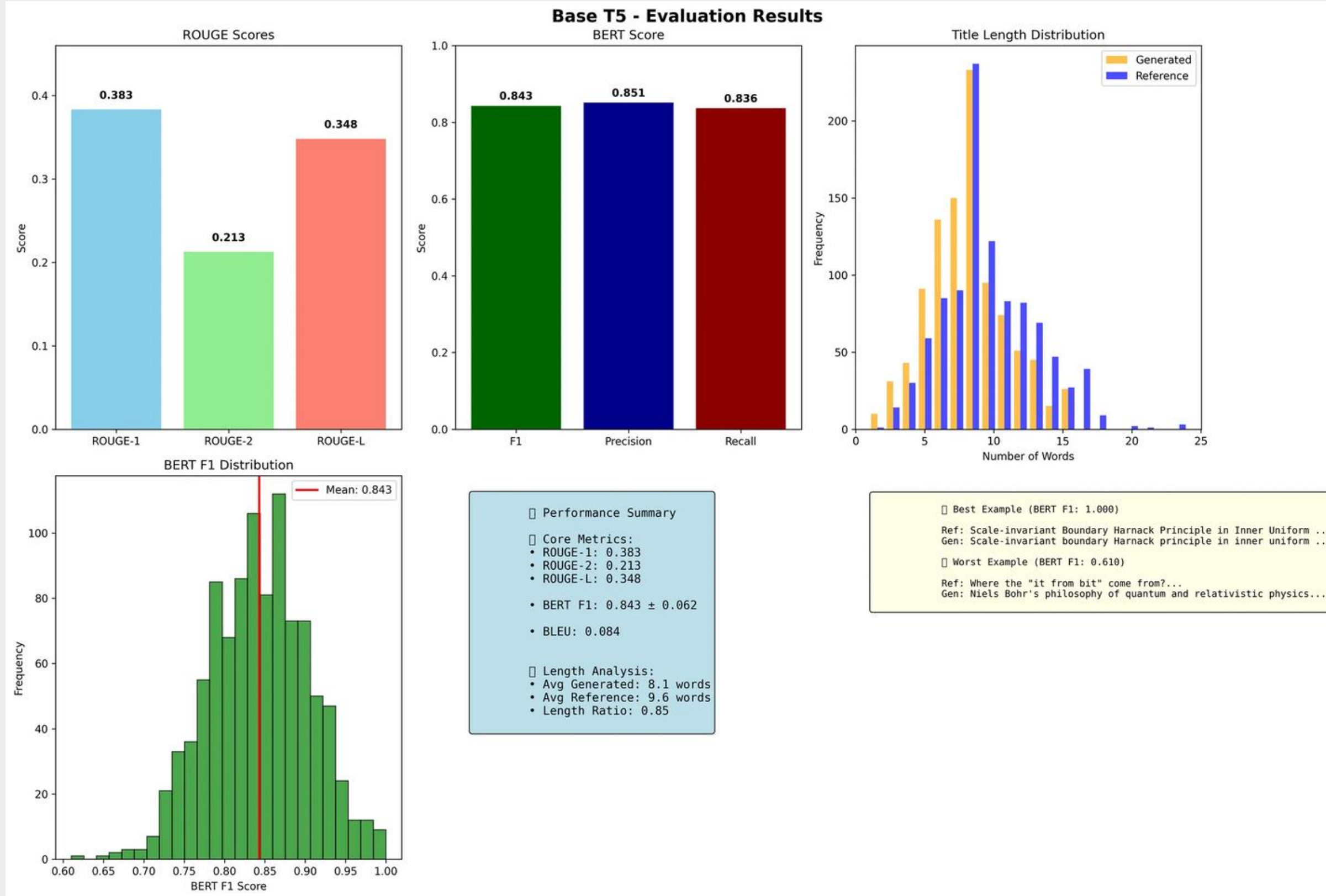
    # 학습 설정
    training_args = TrainingArguments(
        output_dir=CONFIG["output_dir"],
        num_train_epochs=5,
        per_device_train_batch_size=8,
        learning_rate=3e-5,
        # DCG 모델과 동일한 설정 사용
    )

    # 트레이너 초기화 및 학습
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset
    )
```

**목적: 표준 T5
모델의 성능을
베이스라인으로 설정**

**의도: 초록에서
제목 생성 태스크의
기본 성능 측정**

* Base T5



ROUGE-1 : 0.383
ROUGE-2 : 0.213
ROUGE-L : 0.348
BERT Score F1 : 0.843

* Dynamic Context Gates (DCG)

1. DCG 기본 개념

목적: 소스 컨텍스트(인코더)와 타겟 컨텍스트(디코더) 간의 동적 균형 조절

원리: 단어 생성 시 각 컨텍스트의 기여도를 게이트로 제어

응용: 번역, 요약, 제목 생성 등 시퀀스-투-시퀀스 태스크

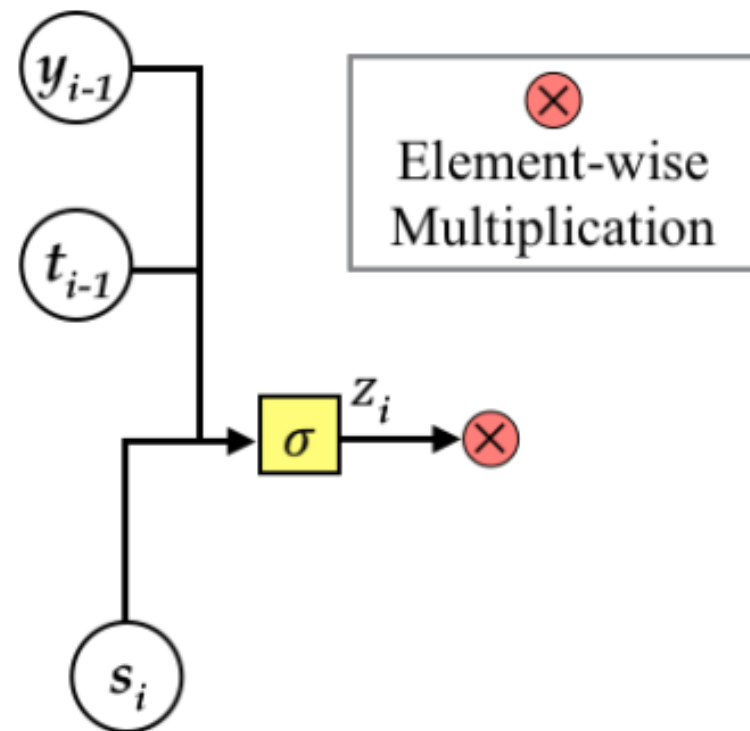
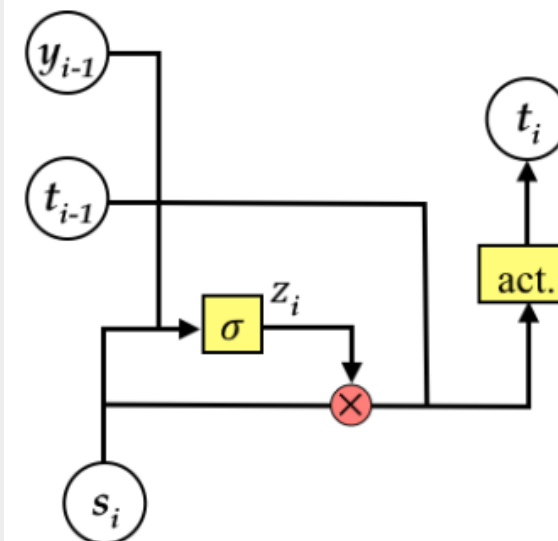
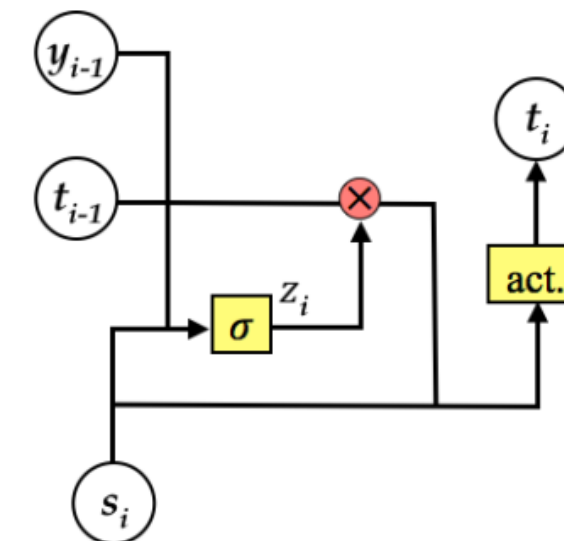


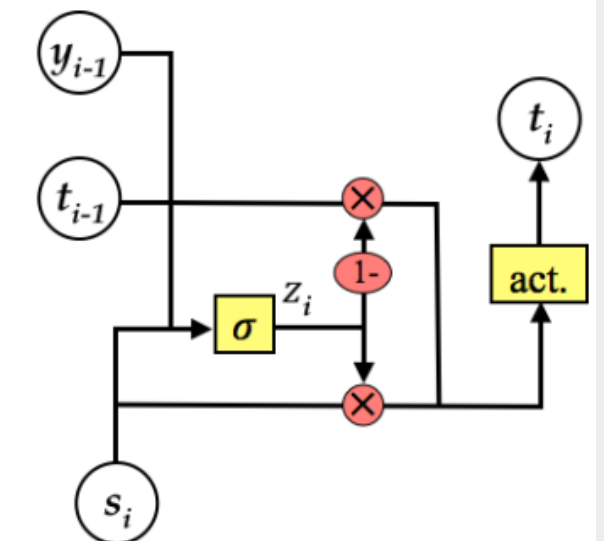
Figure 3: Architecture of context gate.



(a) Context Gate (*source*)



(b) Context Gate (*target*)



(c) Context Gate (*both*)

Figure 4: Architectures of NMT with various context gates, which either scale only one side of translation contexts (*i.e.*, source context in (a) and target context in (b)) or control the effects of both sides (*i.e.*, (c)).

참고 논문 : Context Gates for Neural Machine Translation, Tu et al. (2017)

RNN 기반 NMT에서 소스/타겟 컨텍스트 기여도 동적 제어 메커니즘 최초 제안

* Dynamic Context Gates (DCG)

동적 게이트 계산

$$gate = sigmoid([c; h_t] \cdot W + b)$$

- c : 인코더 컨텍스트 벡터
- h_t : 디코더 현재 히든 스테이트
- W, b : 학습 가능한 파라미터

컨텍스트 융합

$$c'_t = gate * c + (1 - gate) * h_t$$

- c'_t : 융합된 컨텍스트 (다음 레이어 입력)

✅ 게이트 적용 위치

전체가 아닌, T5의 각 T5Block 내부에 게이트를 삽입.
셀프 어텐션 출력과 블록 입력(또는 디코더 은닉 상태)
간의 정보를 벡터 단위로 동적으로 조절.

✅ 구현 용이성

PyTorch 기반 서브클래싱으로 T5Block의
forward() 메소드만 수정해 구현.
기존 Trainer 로직 대부분 재사용 가능,
실험 설계와 성능 비교가 용이.

* AdvancedDCG T5

고급 DCG 메커니즘 다중 레이어 DCG 적용

```
# advanced_dcg_train.py (이전 dcg_train_2.py)
class AdvancedDynamicContextGates(nn.Module):
    def __init__(self, hidden_size, num_heads=4, dropout_rate=0.1):
        # 멀티헤드 어텐션 컨텍스트 이해
        self.context_attention = nn.MultiheadAttention(
            embed_dim=hidden_size,
            num_heads=num_heads,
            dropout=dropout_rate,
            batch_first=True
        )
```

```
# 내용어 탐지 모듈
self.content_detector = nn.Sequential(
    nn.Linear(hidden_size, hidden_size // 2),
    nn.ReLU(),
    nn.Dropout(dropout_rate),
    nn.Linear(hidden_size // 2, 1),
    nn.Sigmoid()
)

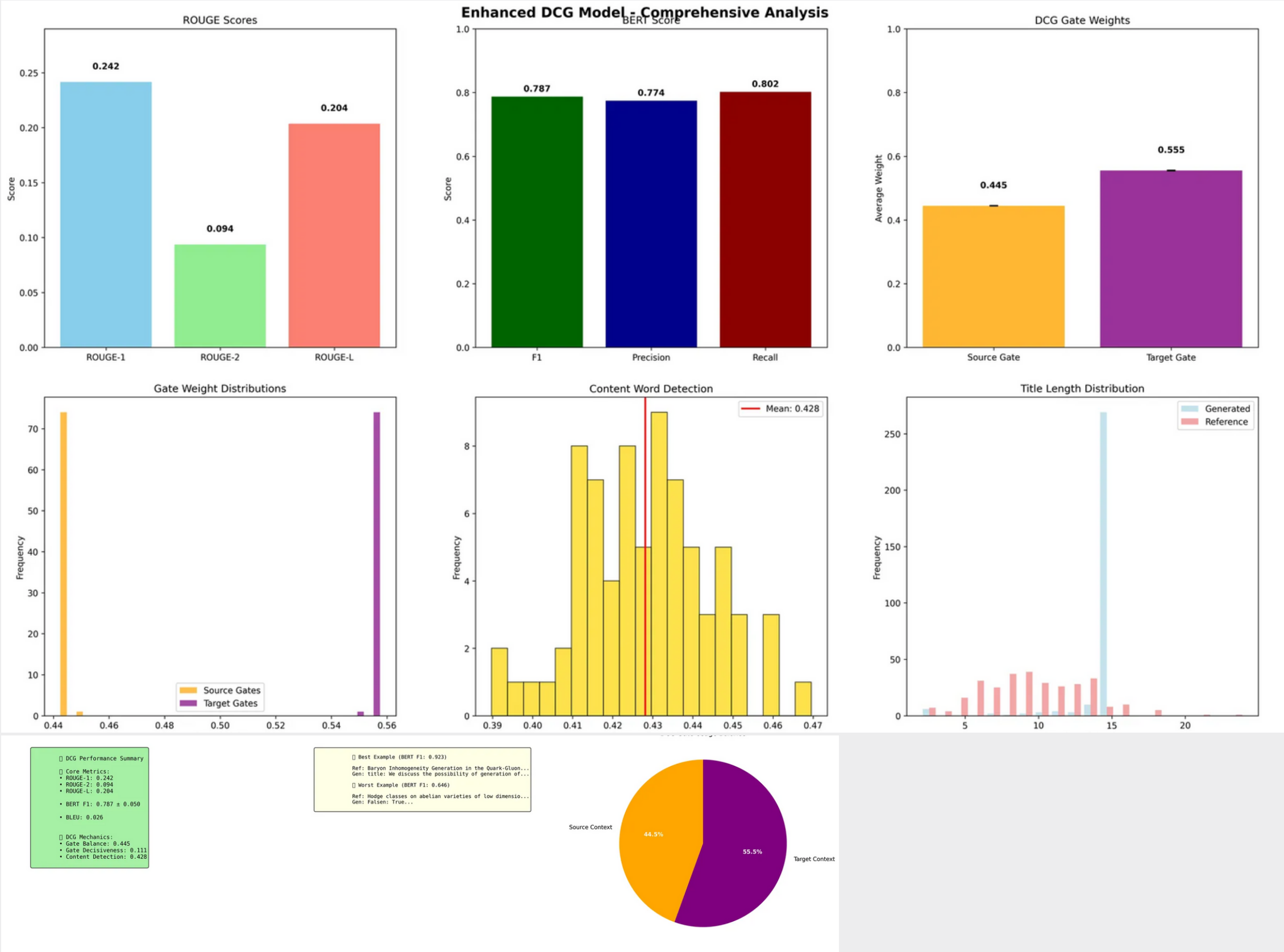
# 위치 인식 임베딩
self.position_embedding = nn.Embedding(128, hidden_s

# 적응형 임계값 학습
self.adaptive_threshold = nn.Parameter(torch.tensor(
```

```
# advanced_dcg_train.py
class EnhancedT5WithDCG(T5ForConditionalGeneration):
    def __init__(self, config):
        # 전략적으로 선택된 디코더 레이어에 DCG 모듈 추가
        dcg_layers = [0, 2, 4] if config.num_decoder_layers >= 6 else [0, 2]

        self.dcg_modules = nn.ModuleDict({
            str(layer_idx): AdvancedDynamicContextGates(
                hidden_size=config.d_model,
                num_heads=4,
                dropout_rate=CONFIG["gate_dropout"]
            ) for layer_idx in dcg_layers
        })
```

* AdvancedDCG T5



ROUGE-1 : 0.242
ROUGE-2 : 0.094
ROUGE-L : 0.204
BERT Score F1 : 0.787

* Simple T5

단순화된 DCG 메커니즘 점진적 DCG 활성화

```
# simple_dcg_train.py (이전 rp_dcg_train_3.py)
```

```
class SimpleDynamicContextGates(nn.Module):
```

```
    def __init__(self, hidden_size, dropout_rate=0.1):
```

```
        # 단순화된 소스/타겟 게이트
```

```
        self.source_gate = nn.Sequential(
```

```
            nn.Linear(hidden_size * 3, hidden_size // 2),
```

```
            nn.ReLU(),
```

```
            nn.Dropout(dropout_rate),
```

```
            nn.Linear(hidden_size // 2, 1),
```

```
            nn.Sigmoid()
```

```
        )
```

```
        self.target_gate = nn.Sequential(
```

```
            nn.Linear(hidden_size * 3, hidden_size // 2),
```

```
            nn.ReLU(),
```

```
            nn.Dropout(dropout_rate),
```

```
            nn.Linear(hidden_size // 2, 1),
```

```
            nn.Sigmoid()
```

```
        )
```

```
# simple_dcg_train.py
```

```
class ImprovedT5WithDCG(T5ForConditionalGeneration):
```

```
    def __init__(self, config):
```

```
        # 단일 DCG 모듈 (마지막 레이어에만 적용)
```

```
        self.dcg_module = SimpleDynamicContextGates(
```

```
            hidden_size=config.d_model,
```

```
            dropout_rate=CONFIG["gate_dropout"]
```

```
        )
```

```
        # DCG 점진적 활성화 파라미터
```

```
        self.register_buffer('training_step', torch.tensor(0))
```

```
        self.dcg_warmup_steps = CONFIG["dcg_warmup_steps"]
```

```
    def get_dcg_alpha(self):
```

```
        """DCG 활성화 비율 계산"""
```

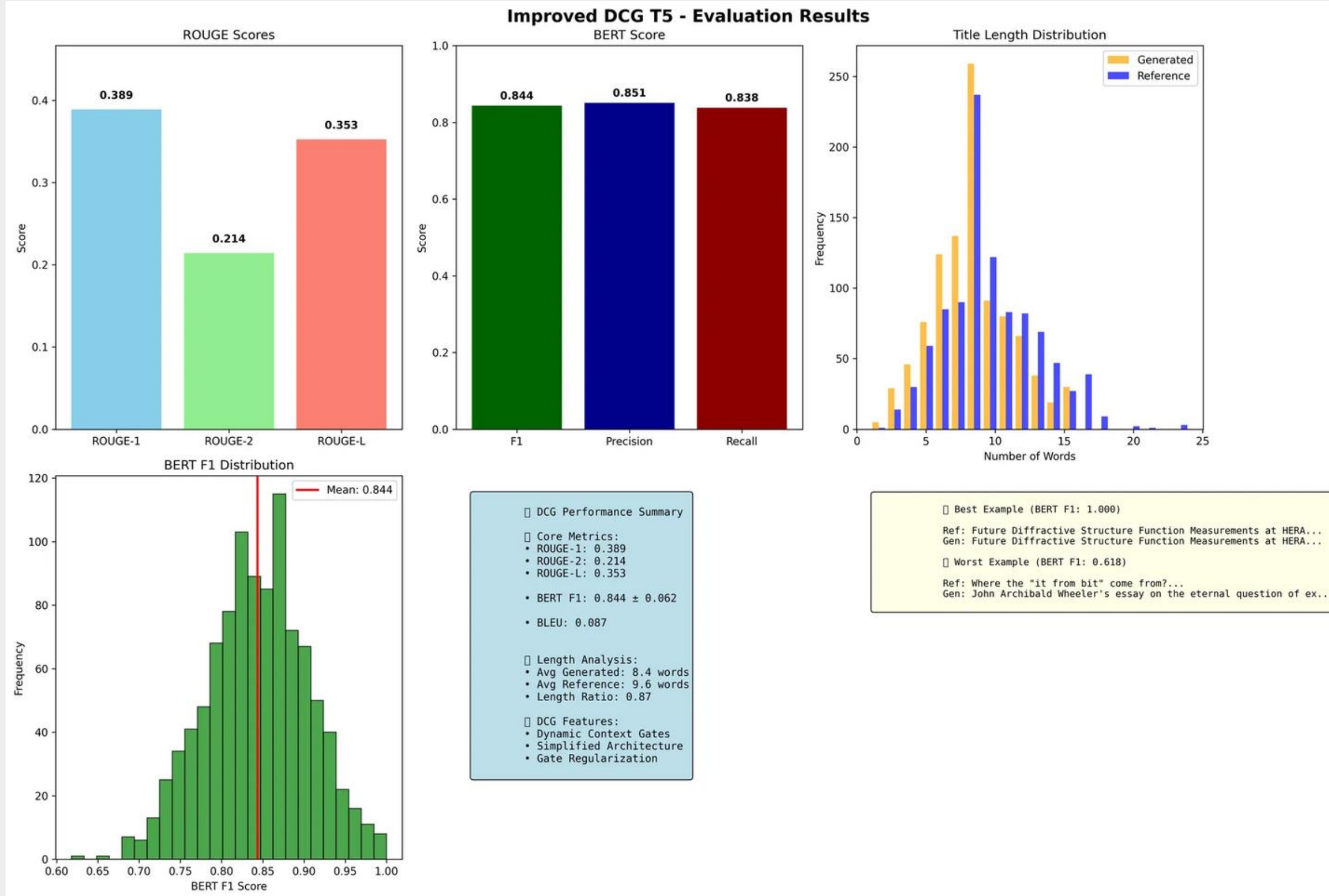
```
        if self.training:
```

```
            alpha = min(1.0, self.training_step.float() / self.dcg_warmup_steps)
```

```
            return alpha
```

```
        return 1.0
```


* Simple T5



ROUGE-1 : 0.389
ROUGE-2 : 0.214
ROUGE-L : 0.353
BERT Score F1 : 0.844

✳ Best T5 Model?

모델 별 성능 비교

Model	ROUGE-1	ROUGE-2	ROUGE-L	BERT Score F1
Base T5	0.383	0.213	0.348	0.843
AdvancedDCG T5	0.242	0.094	0.204	0.787
SimpleDCG T5	0.389	0.214	0.353	0.844

SimpleDCG T5 > Base T5 > AdvancedDCG T5
단순화된 DCG 모델이 가장 높은 성능 달성

성능 순위

길이 분석

모델	생성 제목 평균 길이	참조 제목 평균 길이	길이 비율
Base T5	8.1	9.6	0.85
SimpleDCG T5	8.4	9.6	0.87

AdvancedDCG의 성능 저하
SimpleDCG의 성공

* 실제 생성 시연

데모 시연

- Base T5

* 실제 생성 시연

데모 시연

- Simple T5

* Ab2Ti 프로젝트의 결론

DCG의 T5 적용 성공

두 가지 DCG 구현 방식

- RNN 기반 모델에서만 적용되던 DCG를 Transformer 기반 T5 모델에 성공적으로 적용
- AdvancedDCG: 복잡한 게이트 메커니즘
- SimpleDCG: 단일 디코더 레이어 적용, 점진적 활성화

성능 향상 확인

- BLEU 점수: 0.087 (8.7%)
- BERT F1: 0.844 ± 0.062
- ROUGE-1: 0.389 / ROUGE-2: 0.214 / ROUGE-L: 0.353

핵심

- 본 연구는 논문 초록에서 제목 생성이라는 구체적인 NLP 태스크에서 DCG 메커니즘의 효과를 입증
- 특히 학술 논문의 제목 생성 시 초록의 전문적 내용을 정확히 반영하는 것이 중요한데, DCG는 이러한 "내용 충실성(content faithfulness)"을 향상시키는 데 효과적임을 확인

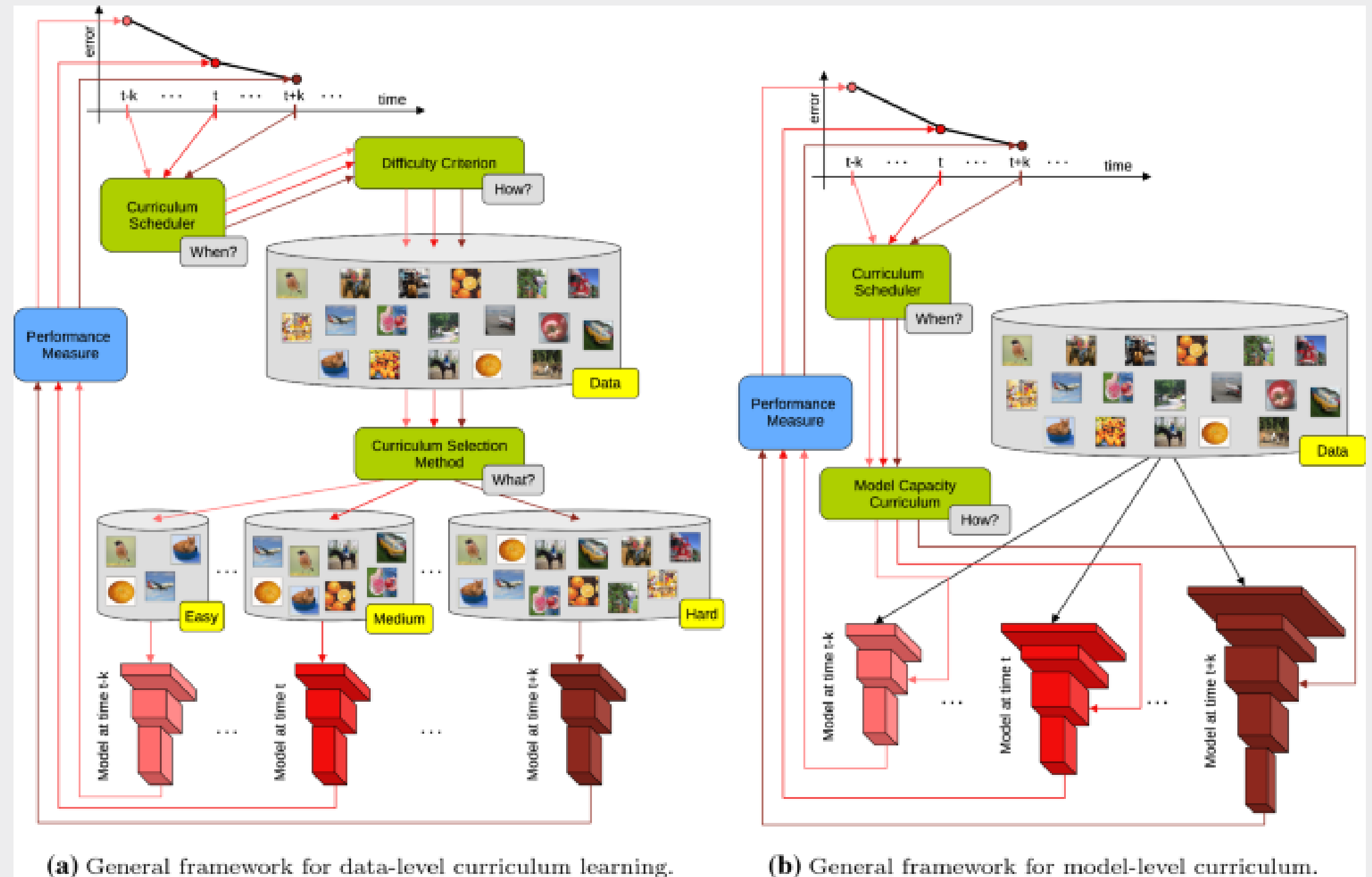
* Ab2Ti 프로젝트의 개선점

알고리즘 측면



쉬운 데이터 부터 학습하는 curriculum learning 알고리즘 적용

1. 게이트 융합 메커니즘 개선
2. 커리큘럼 러닝 적용



* Ab2Ti 프로젝트의 개선점

미니 프로젝트 설명

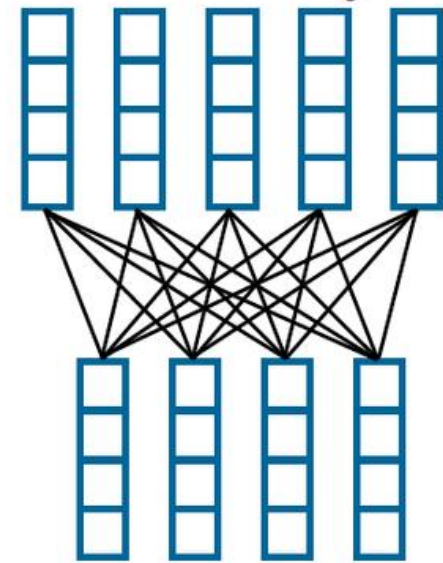
평가 메트릭 추가



1. 학습 중 평가 메트릭 부재
2. 정성적 평가 부족
3. 평가 메트릭과 학습 과정의 분리

Bert-Score ,METEOR Score

Pairwise Cosine Similarity

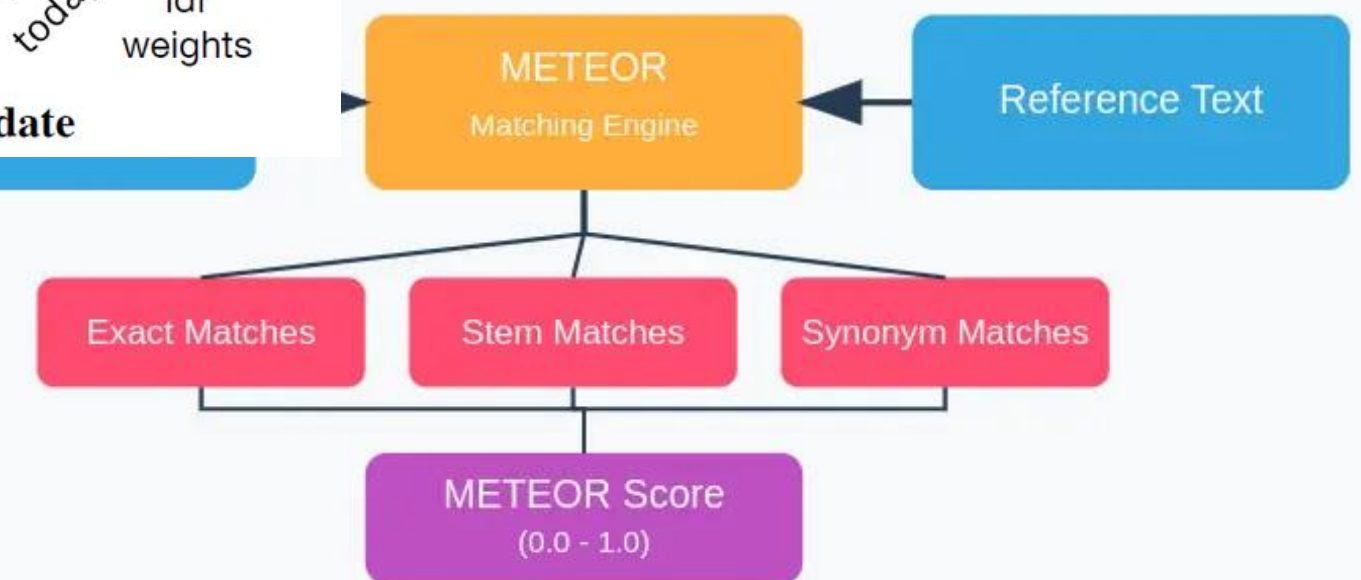


Maximum Similarity

the	0.713	0.597	0.428	0.408	1.27
weather	0.462	0.393	0.515	0.326	7.94
is	0.635	0.858	0.441	0.441	1.82
cold	0.479	0.454	0.796	0.343	7.90
today	0.347	0.361	0.307	0.913	8.88
	it	is	freezing	today	idf weights

Zhang et al., ICLR 2020, BERTScore.

Evaluation Framework



Banerjee & Lavie, ACL 2005, METEOR.

감사합니다.

<https://github.com/minuum/NLP1>

논문 제목 생성기
Ab2Ti (Abstract to Title)